

Conditions

- ability to ask questions and answer those questions in order to decide which code to execute

built-in Syntax

$>$, $>=$ greater than, greater than or equal to
 $<$, $<=$ less than, less than or equal to
 $==$, $!=$ equal to, not equal to (note: $=$ means assignment)

To ask questions using the above Syntax: if Statements

ex. Comparing values to make a decision based on the values (assume $[x] = 1$, $[y] = 2$)

$x = \text{int}(\text{input}(\text{"what's x? "}))$

What's x? [x]

$y = \text{int}(\text{input}(\text{"what's y? "}))$

What's y? [y]

if $x < y$:

x is less than y

... print("x is less than y")

Boolean expression:

if $x > y$:

Simply a question with a yes/no

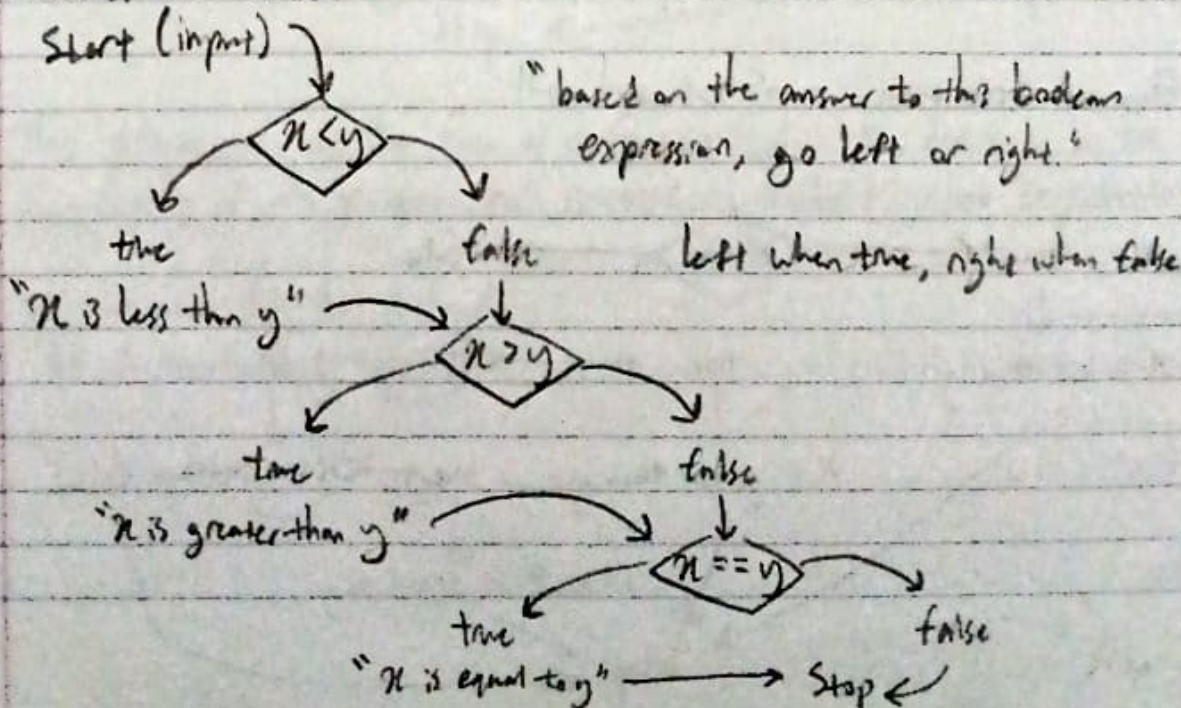
... print("x is greater than y")

or true/false question

if $x == y$:

... print("x is equal to y")

flow of execution:



Whether or not the expression is true or false, the code travels down the program, onto the next statement.

It's not enough for the code to be correct and do what is intended. In the long run, programs have to be well-designed as well.

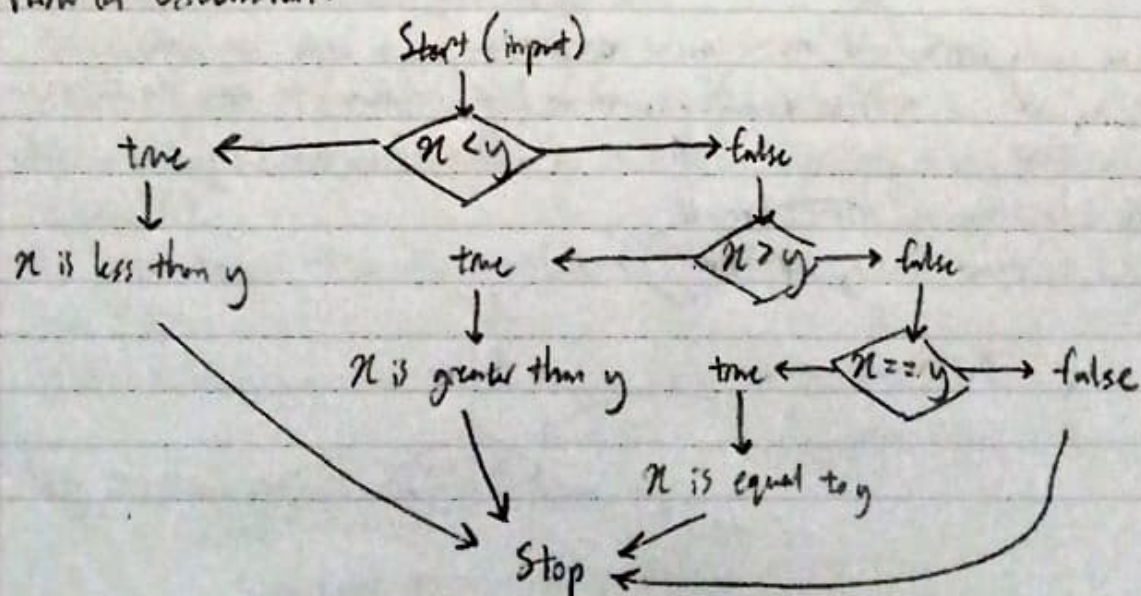
From the example: the code does not need to continue to run if we already got our result.

elif: Conjunction to else if, allows us to ask a question that takes into account whether or not a previous question had a true/false answer

to simplify the earlier code, make the earlier conditions potentially and mutually exclusive; don't keep answering questions once we get back a true answer

ex.	<code>x = int(input("What's x? "))</code>	What's x? [x]
	<code>y = int(input("What's y? "))</code>	What's y? [y]
	<code>if x < y:</code>	x is less than y
	<code>... print("x is less than y")</code>	
	<code>elif x > y:</code>	
	<code>... print("x is greater than y")</code>	
	<code>elif x == y:</code>	
	<code>... print("x is equal to y")</code>	

Flow of execution:

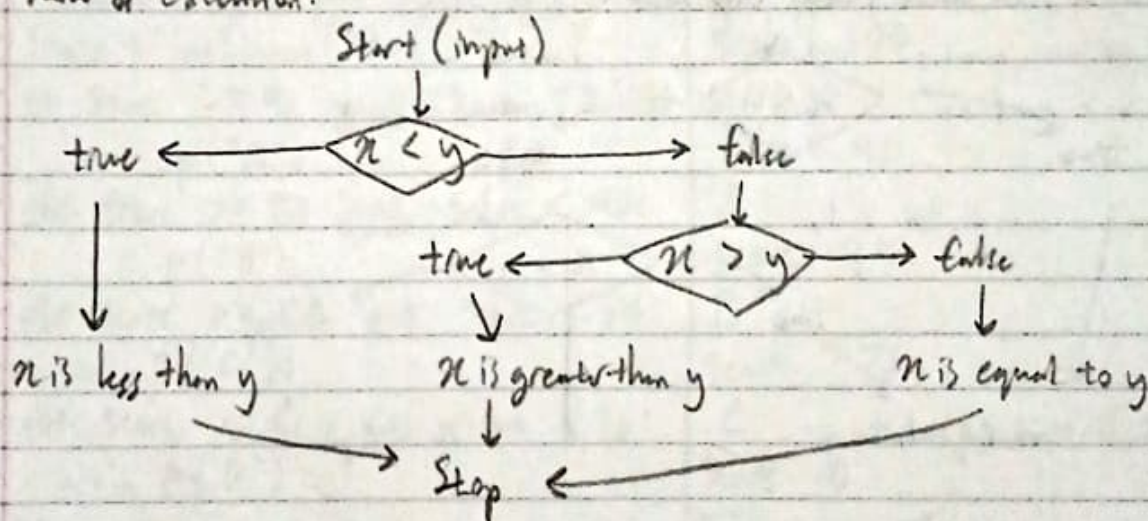


Attune for example on this page, $[x]$ and $[y]$ have values written on terminal

looking at the example, if $x > y$ and $x < y$ are both false, we don't need to have an `elif` (else if) for $x == y$. Add "else"

ex.	<code>x = int(input("what's x? "))</code>	what's x? 1
	<code>y = int(input("what's y? "))</code>	what's y? 2
	<code>if x < y:</code>	x is less than y
	<code>print("x is less than y")</code>	what's x? 2
	<code>elif x > y:</code>	what's y? 1
	<code>print("x is greater than y")</code>	x is greater than y
	<code>else:</code>	what's x? 2
	<code>print("x is equal to y")</code>	what's y? 2
		x is equal to y

flow of execution:



The progression of the flow of execution shows the decrease in the complexity of the program and increase in readability and simplification.

or statement for this example:

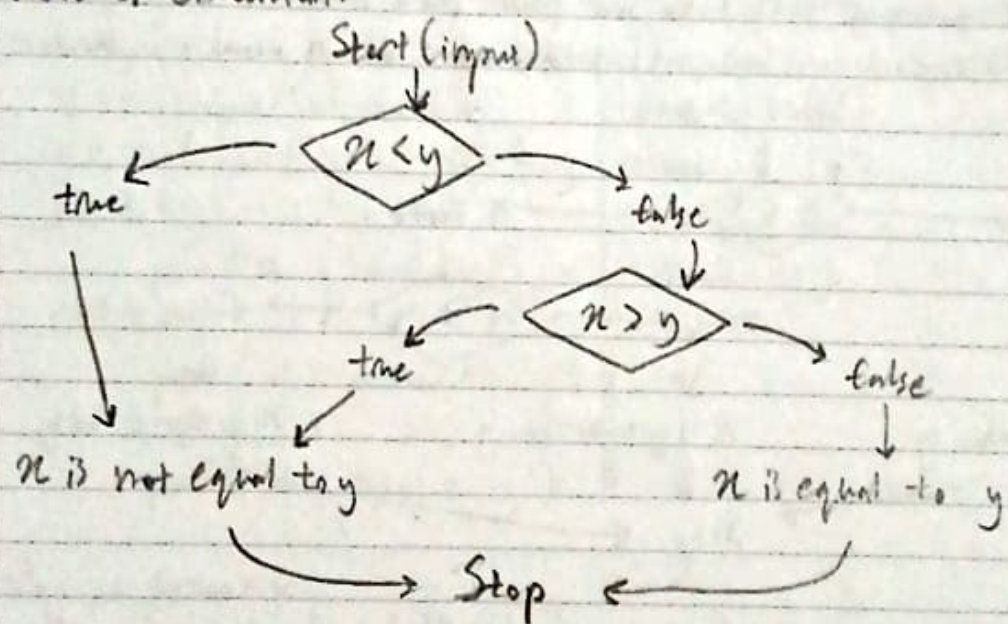
Is x equal to y or not?

ex. assume $[x]$ and $[y]$ have values written on terminal

```
x = int(input("what's x? "))
y = int(input("what's y? "))
if x < y or x > y:
    ... print("x is not equal to y")
else:
    ... print("x is equal to y")
```

```
What's x? 1
What's y? 2
x is not equal to y
What's x? 2
What's y? 1
x is not equal to y
What's x? 1
What's y? 1
x is equal to y
```

Flow of execution:



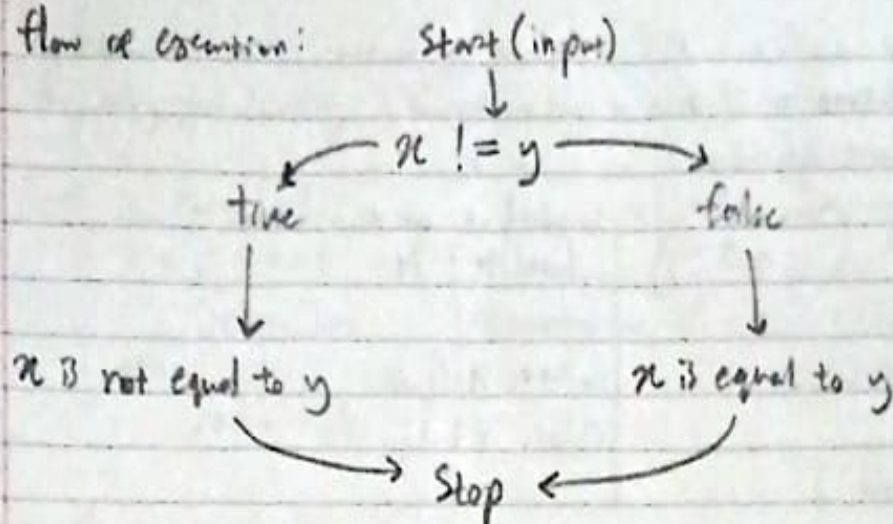
Instead of asking if $x < y$ or $x > y$, I can just ask if $x == y$, else x is not equal to y and vice versa.

ex. assume $[x]$ and $[y]$ have values written on terminal

```
x = int(input("what's x? "))
y = int(input("what's y? "))
if x != y:
    ... print("x is not equal to y")
else:
    ... print("x is equal to y")
```

```
What's x? 1
What's y? 2
x is not equal to y
What's x? 1
What's y? 1
x is equal to y
```


Flow of execution:



and Statement

What if a value is between some value and some other value?

ex. assume "p" is the print function, [S] is user input with the value in terminal

Score = int(input("score: "))	Score: 100
if Score $\geq$ 90 and Score $\leq$ 100:	A
... p("A")	Score: 91
elif Score $\geq$ 80 and Score $<$ 90:	A
... p("B")	Score: 89
elif Score $\geq$ 70 and Score $<$ 80:	B
... p("C")	Score: 71
elif Score $\geq$ 60 and Score $<$ 70:	C
... p("D")	Score: 0
else:	F
... p("F")	

To simplify these if/elif statements, we can remove the and statements and nest together the score evaluations.

if $90 \leq \text{Score} \leq 100$ $\rightarrow$ if $90 \leq \text{Score} \leq 100$

we can also shorten the statements by having only 1 question.

if Score $\leq$ 90:

elif Score $\leq$ 80:

elif I've already checked if the student's score is greater than 90.

Parity

- refers to whether a number is even or odd
- utilizes the modulo operator

ex. assume $[x]$ is a value written in terminal

```
x = int(input("what's x? "))
if x % 2 == 0:
    print("even")
else:
    print("odd")
```

what's x? 4
even
what's x? 3
odd

If we want to use this functionality elsewhere in our code, we can define a new function. We will also use Boolean Values. Boolean values, or Bool, are a new datatype that can only be True or False.

ex. assume $[x]$ is a value written in terminal

```
def main():
    x = int(input("what's x? "))
    if is_even(x):
        print("even")
    else:
        print("odd")
```

what's x? 4
even
what's x? 3
odd

```
def is_even(n):
    if n % 2 == 0:
        return True
    else:
        return False
```

main()

Boolean Expression

I can reuse this function, since it can return values to other functions, to evaluate if a number is even/odd.

All that is needed for a Conditional is a decision that is either true or false.

Pythonic

- the way you do things in Python

```
ex def is_even(n):  
    ... if n % 2 == 0  
    ...     return True  
    ... else:  
    ...     return False
```

} return True if $n \% 2 == 0$ else False

to shorten it further:

```
ex def is_even(n):  
    ... return n % 2 == 0
```

Because $n \% 2 == 0$ is a Boolean expression alone, we no longer need to add if/else statements. Basically:

$n \% 2 == 0$, True. $n \% 2 != 0$, False.
and the return function will return either True or False.

Match (Similar to "Switch" in other languages)

- expresses the same idea more compactly

```
ex. name = input("your name: ")  
match name:  
    ... case "Harry":  
    ...     print("Gryf")  
    ... case "Hermione":  
    ...     print("Gryf")  
    ... case "Ron":  
    ...     print("Gryf")  
    ... case "Draco":  
    ...     print("Slyth")  
    ... case _:  
    ...     print("who?")
```

your name: Harry
Gryf
your name: Draco
Slyth
your name: Pank
who?

In the context of "match", the underscore represents whatever case has not been handled.

The keyword "Match" helps organize and perform pattern matching, comparing a given value against a series of patterns defined in case blocks. Simplifying it:

ex. name = input("your name: ")	your name: Ron
match name:	Gryf
.... case "Harry" "Hermione" "Ron":	your name: Draco
.... print("Gryf")	Slyth
.... case "Draco":	your name: Pook
.... print("Slyth")	who?
.... case _:	
.... print("who?")	